Conway's Game of Life



Background

Origins of Cellular Automata

Cellular Automata was born and raised during WWII and the post-war period at Los Alamos National Laboratory with Stanisław Ulam & John von Neumann working on simulating crystal growth, self-replicating systems, and eventually fluid dynamics.



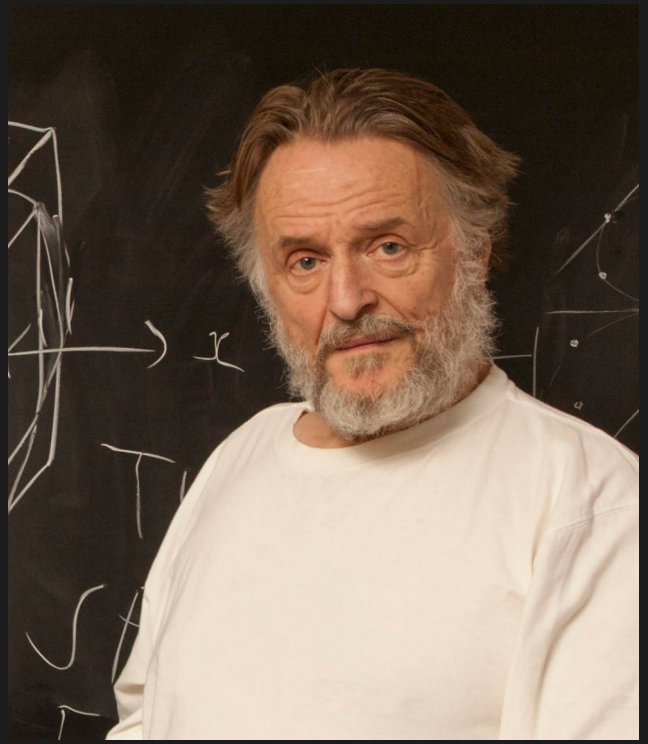
Stanisław Ulam



John Von Neumann

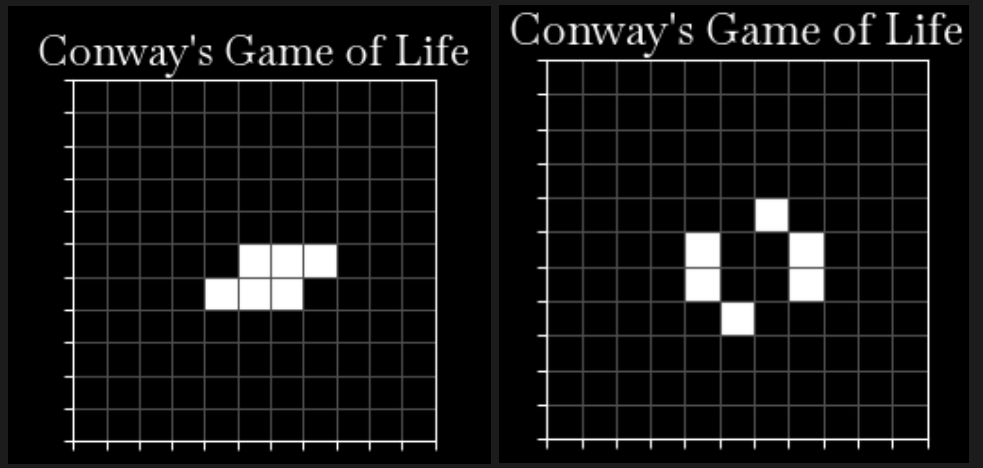
Conway's Game of Life

- John Conway in 1968, read about Neumann's Work in self-replication.
- Conway wanted to make a more simple (less states) and unpredictable automata, under the philosophy that "if you couldn't predict the outcome, it probably can do anything."
- Conway told Martin Gardner about his Game of Life and was published in the *Scientific American* October 1970.



Rules of Conway's Game

- If a dead (false) cell is surrounded by exactly three live cells, it is born (becomes true)
- If a live (true) cell is surrounded by more than three live cells or less than two live cells, it dies (becomes false)
- Under any other condition the cell remains constant

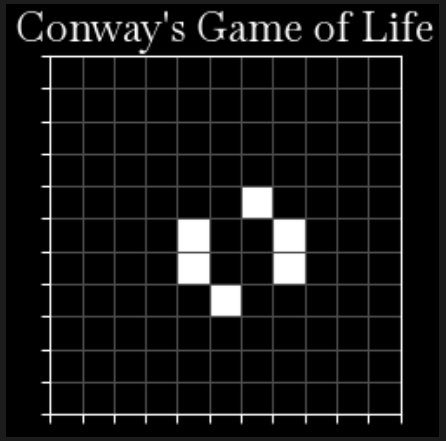


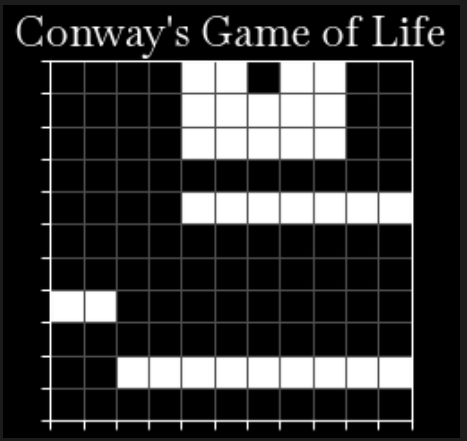
Under normal rules of Conway's Game, the game would switch between these two states forever.

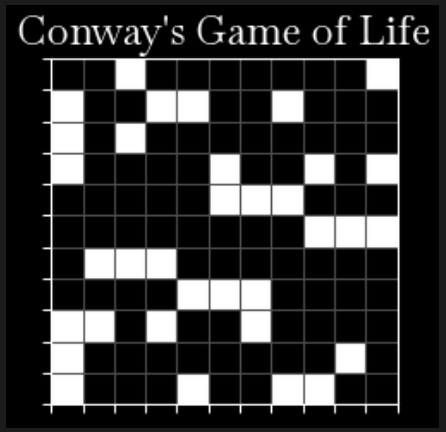
Design

Settings Storage

- Used JSON file because python library and easy indexing of individual elements.
- Stored following data:
  - Height and width of grid
  - Radius for live and dead cells when looking at adjacencies
  - Overpopulation and under-population thresholds
  - Number of cells required for reproduction



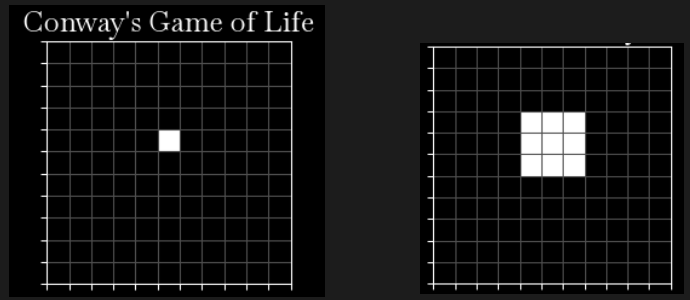




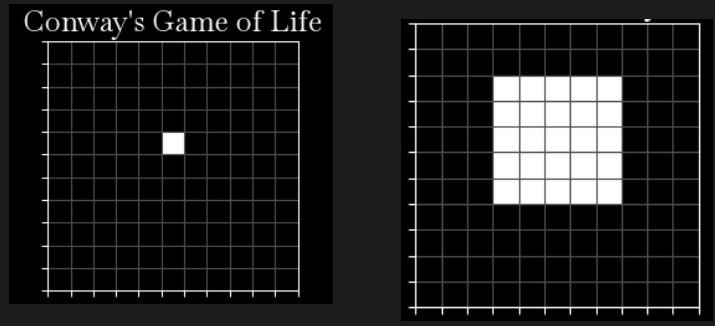
Three different rule settings with the same initial conditions

Adjacency Function

The first thing I did was make a function that was given the coordinates of the original cell and a search radius around it that will be considered adjacent. The function returns an integer of the number of live cells within the search radius (not including the given cell).



Conway Search Radius = 1



Conway Search Radius = 2

Iteration Function

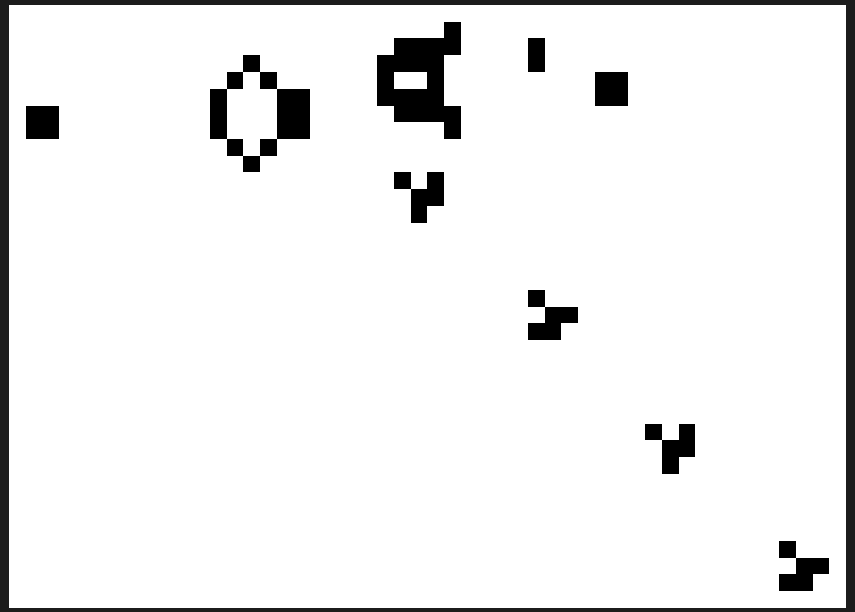
The iteration function simply runs the adjacency function on every cell, and changes the game board based on the outputs of the adjacency function.

```
def iterate_conway():
    global MAP, STORED_MAP, DIFFERENTIAL_MAP, SEARCH_CELLS
    map = MAP.copy()

    set_search_area()
    for (row, col), value in np.ndenumerate(MAP):
        #print(f"row, col: {row}, {col}\nValue: {value}")
        if SEARCH_CELLS[row, col] == True:
            if value == True:
                if (cell_adjacencies(MAP, RADIUS_LIVE_CELL, row, col) <= UNDERPOP_TSH) or (cell_adjacencies(MAP, RADIUS_LIVE_CELL, row, col) >= OVERPOP_TSH):
                    #print("Killed")
                    map[row, col] = False
            else:
                if cell_adjacencies(MAP, DEAD_CELL_RADIUS, row, col) == REPRODUCE_ADJ:
                    #print("Born")
                    map[row, col] = True
    STORED_MAP = np.copy(MAP)
    #print(STORED_MAP)
    MAP = np.copy(map)
```

Pausing and Updating

I used a global pause variable and added a button to the matplotlib figure to have pause functionality. Then I made a two functions: pause_unpause (simply inverts the global pause variable) and update. Update is tied to a timer that executes in fixed intervals. If Conway's Game is paused then update does nothing. If Conway's Game is not paused then it iterates and refreshes the matplotlib figure.



Input

Mouse Input

I made mouse input using mpl_connect. The grid_click function returns without doing anything if a click was when the game was not paused or was not in the top left figure. If the game is paused and the click is on the top left figure, then it will invert the cell clicked.

CSV Input

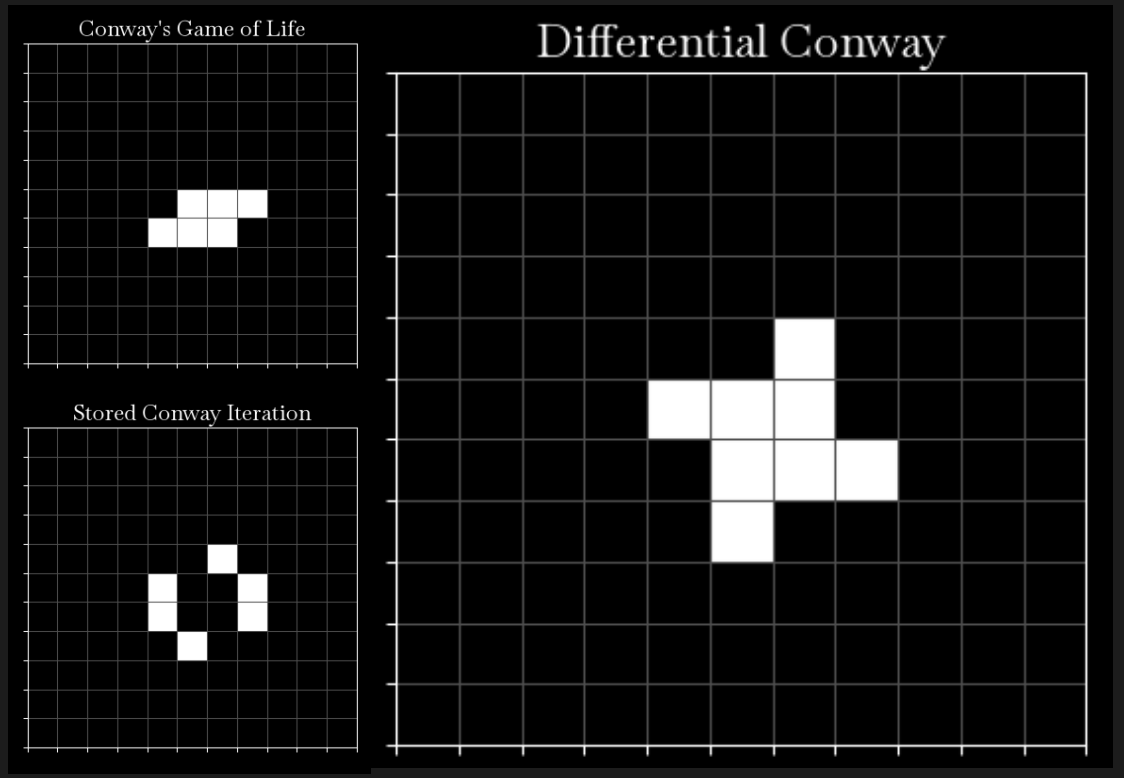
The formatting of the CSV input is the coordinates of cells that should be alive. The program takes the CSV and makes all cells listed alive. There's a boolean variable whether or not to use the CSV

Optimizations



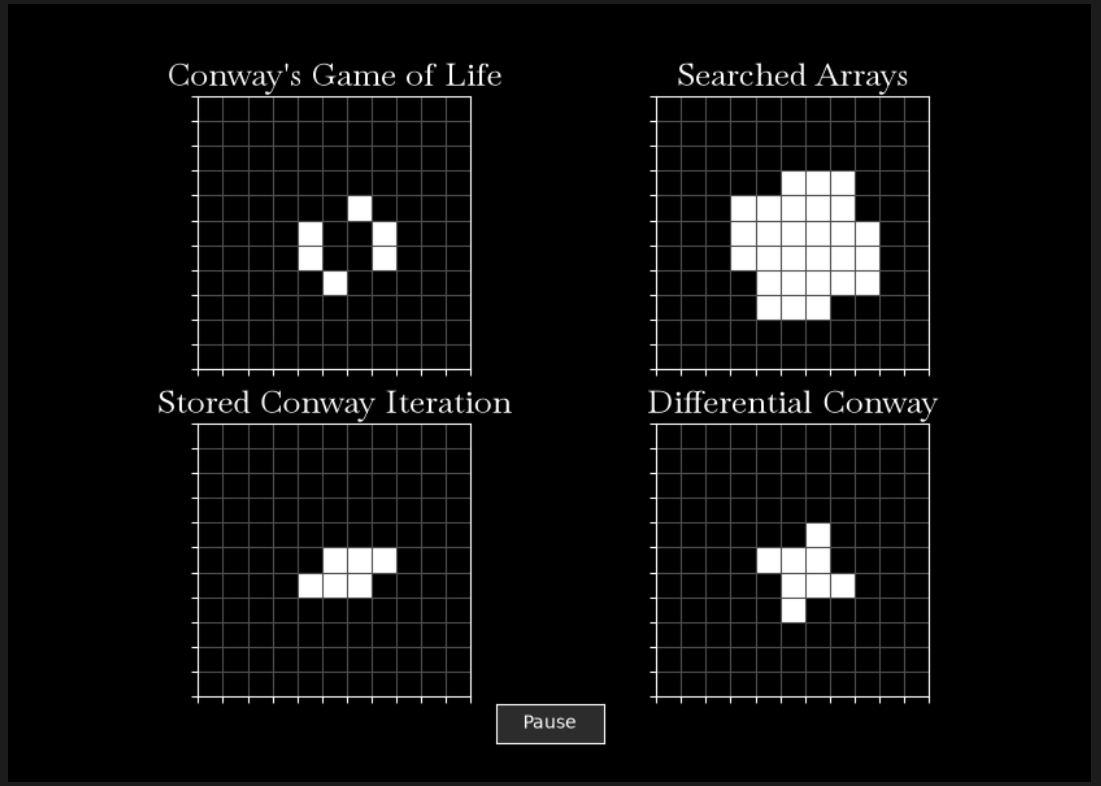
Stored & Differential Array

- Searching through every individual cell is a waste of time.
- We should store the previous iteration and create a differential array that tracks what has changed between the previous and current iteration. That's what I did.



Search Array

- From the differential array we add a radius around each true element of the differential array.
- When we are going through the iteration function we only check adjacencies from elements that are true in the search array reducing the amount of iterations by (2(radius)+1)^2 for each element not in the search array



Thank you

